

## ASSIGNMENT – 11

**NAME : M.AKSHAY PATEL**

**HALL TICKET NO : 2303A51745**

**BATCH : 11**

### Question :

Hyperledger Fabric – Comparative Exploration Objective To study and practically explore Hyperledger

Fabric by setting up a basic Fabric test network, deploying a simple chaincode, and comparing Hyperledger Fabric with a public blockchain (Ethereum) based on architecture, consensus, privacy, performance, and use cases.

Requirements :Software Requirements

Operating System: Ubuntu 20.04 / Windows (with WSL) / macOS • Docker & Docker Compose • Git • cURL • Node.js (v16+) or Go (for chaincode) • Hyperledger Fabric binaries and samples Hardware Requirements • Minimum 8 GB RAM • Internet connection

### CODE :

```
import tkinter as tk from tkinter
import ttk, messagebox import
random import time

class DeFi:      def
__init__(self):
self.balances = {}

    def gas(self):
        return round(random.uniform(0.0001, 0.001), 6)

    def delay(self):
time.sleep(0.3)

    def deposit(self, user, amount):
if amount <= 0:
    return "Invalid amount"
```

```

        self.delay()
gas = self.gas()

        if user not in self.balances:
self.balances[user] = 0

        self.balances[user] += amount    return
f"Deposit Successful\nGas Fee: {gas}"

```

```

def withdraw(self, user, amount):
if user not in self.balances:
return "User not found"

```

```

        if self.balances[user] < amount:
return "Insufficient balance"

```

```

        self.delay()
gas = self.gas()

```

```

        self.balances[user] -= amount    return
f"Withdraw Successful\nGas Fee: {gas}"

```

```

def balance(self, user):
    return self.balances.get(user, 0)
class App:
    def __init__(self, root):    self.defi
= DeFi()    self.root = root
self.root.title("DeFi Tab Interface")
self.root.geometry("500x350")
notebook = ttk.Notebook(root)

```

```
notebook.pack(expand=True,  
fill="both")
```

```
    # Tabs    self.deposit_tab =  
tk.Frame(notebook)    self.withdraw_tab  
= tk.Frame(notebook)    self.balance_tab  
= tk.Frame(notebook)
```

```
    notebook.add(self.deposit_tab, text="Deposit")  
notebook.add(self.withdraw_tab, text="Withdraw")  
notebook.add(self.balance_tab, text="Balance")
```

```
    # ----- Deposit Tab -----
```

```
    tk.Label(self.deposit_tab, text="User").pack(pady=5)  
self.d_user = tk.Entry(self.deposit_tab)  
self.d_user.pack()
```

```
    tk.Label(self.deposit_tab, text="Amount").pack(pady=5)  
self.d_amount = tk.Entry(self.deposit_tab)  
self.d_amount.pack()
```

```
    tk.Button(self.deposit_tab, text="Deposit",  
command=self.deposit).pack(pady=10)
```

```
    self.d_output = tk.Label(self.deposit_tab, text="")  
self.d_output.pack()
```

```
    # ----- Withdraw Tab -----
```

```
tk.Label(self.withdraw_tab, text="User").pack(pady=5)  
self.w_user = tk.Entry(self.withdraw_tab)  
self.w_user.pack()
```

```
tk.Label(self.withdraw_tab, text="Amount").pack(pady=5)
self.w_amount = tk.Entry(self.withdraw_tab)
self.w_amount.pack()
```

```
tk.Button(self.withdraw_tab, text="Withdraw",
command=self.withdraw).pack(pady=10)
```

```
self.w_output = tk.Label(self.withdraw_tab, text="")
self.w_output.pack()
```

```
# ----- Balance Tab -----
```

```
tk.Label(self.balance_tab, text="User").pack(pady=5)
self.b_user = tk.Entry(self.balance_tab)
self.b_user.pack()
```

```
tk.Button(self.balance_tab, text="Check Balance",
command=self.check_balance).pack(pady=10)
```

```
self.b_output = tk.Label(self.balance_tab, text="")
self.b_output.pack()
```

```
def deposit(self):
    try:
        user = self.d_user.get()        amount
        = float(self.d_amount.get())    result =
        self.defi.deposit(user, amount)
        self.d_output.config(text=result)    except:
            messagebox.showerror("Error", "Invalid input")
    def withdraw(self):
        try:
```

```

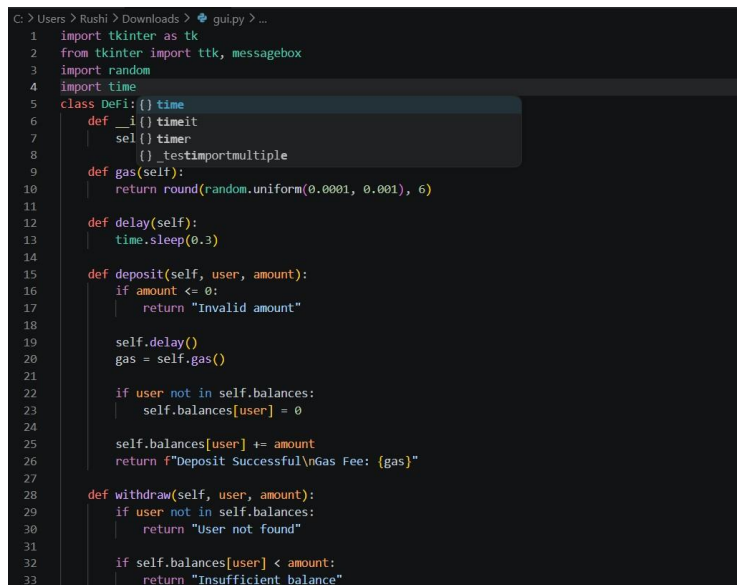
        user = self.w_user.get()        amount =
float(self.w_amount.get())        result =
self.defi.withdraw(user, amount)
self.w_output.config(text=result)        except:
        messagebox.showerror("Error", "Invalid input")

def check_balance(self):        user = self.b_user.get()
balance = self.defi.balance(user)
self.b_output.config(text=f"Balance: {balance} ETH")

if __name__ == "__main__":
    root = tk.Tk()
    app = App(root)
    root.mainloop()

```

## IMPLEMENTATION :



```

C:\Users\Rushi> Downloads > guipy > ...
1  import tkinter as tk
2  from tkinter import ttk, messagebox
3  import random
4  import time
5  class DeFi:() time
6      def __i() timeit
7          sel() timer
8          () _testimportmultiple
9      def gas(self):
10         return round(random.uniform(0.0001, 0.001), 6)
11
12     def delay(self):
13         time.sleep(0.3)
14
15     def deposit(self, user, amount):
16         if amount <= 0:
17             return "Invalid amount"
18
19         self.delay()
20         gas = self.gas()
21
22         if user not in self.balances:
23             self.balances[user] = 0
24
25         self.balances[user] += amount
26         return f"Deposit Successful\nGas Fee: {gas}"
27
28     def withdraw(self, user, amount):
29         if user not in self.balances:
30             return "User not found"
31
32         if self.balances[user] < amount:
33             return "Insufficient balance"

```

```

C:\Users\Rushi> Downloads > gui.py > ...
43 class App:
44     def __init__(self, root):
98         command=self.check_balance).pack(pady=10)
99
100         self.b_output = tk.Label(self.balance_tab, text="")
101         self.b_output.pack()
102
103     def deposit(self):
104         try:
105             user = self.d_user.get()
106             amount = float(self.d_amount.get())
107             result = self.defi.deposit(user, amount)
108             self.d_output.config(text=result)
109         except:
110             messagebox.showerror("Error", "Invalid input")
111
112     def withdraw(self):
113         try:
114             user = self.w_user.get()
115             amount = float(self.w_amount.get())
116             result = self.defi.withdraw(user, amount)
117             self.w_output.config(text=result)
118         except:
119             messagebox.showerror("Error", "Invalid input")
120
121     def check_balance(self):
122         user = self.b_user.get()
123         balance = self.defi.balance(user)
124         self.b_output.config(text=f"Balance: {balance} ETH")
125
126 if __name__ == "__main__":
127     root = tk.Tk()
128     app = App(root)
129     root.mainloop()

```

## OUTPUT :

DeFi Tab Interface

Deposit Withdraw Balance

User  
abc

Amount  
10000000

Deposit

Deposit Successful  
Gas Fee: 0.000831

DeFi Tab Interface

Deposit Withdraw Balance

User  
abc

Amount  
10

Withdraw

Withdraw Successful  
Gas Fee: 0.000297

DeFi Tab Interface

Deposit Withdraw Balance

User  
abd

Check Balance

Balance: 9999990.0 ETH